

SESSION 2 OF 3 · BUILD IT

FROM IDEA TO ORDER

One strategy on the big screen. One screener on your laptop. Every order in the sandbox.

Same code. Two worlds.

UAT · sandbox

- ◆ Fake money, unlimited
- ◆ Same API, same SDK, same dashboards
- ◆ 100 trading ops / sec
- ◆ Today: **every order** goes here
- ◆ Mistakes cost **nothing**

PROD · live

- ◆ Your money
- ◆ Same data, same code, full price history
- ◆ 10 trading ops / sec per IP, static IP rules
- ◆ Today: **read-only**, it feeds your candles and live stream
- ◆ Mistakes cost **money**

```
orders = InitNubraSdk(NubraEnv.UAT, env_creds=True)
data    = InitNubraSdk(NubraEnv.PROD, env_creds=True)
NubraTrader(data)

# today: every order goes here (fake money)
# today: read-only, candles + the live stream
# one word from real consequences. Not today.
```

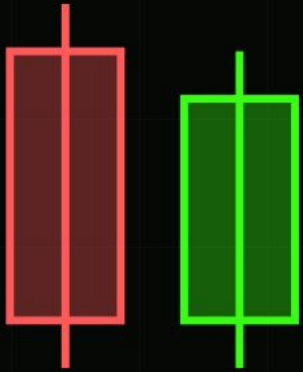
ONE RULE

If you see **NubraEnv.PROD**
near **create_order** today,
raise your hand.

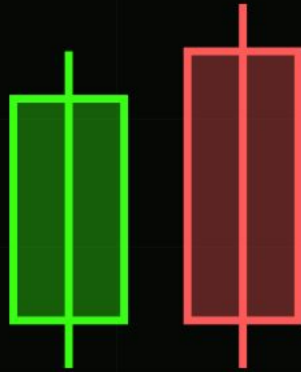


DEMO • THE CANDLE-COLOUR FLIP

Two candles. One rule.



red → green
BUY ATM CALL



green → red
BUY ATM PUT

- ◆ NIFTY, **1-minute** candles, only **completed** ones
- ◆ Nearest expiry, **ATM** strike, **10 lots**
- ◆ On a flip: square off the old leg, buy the new one
- ◆ No stop-loss, no target. A teaching toy, not advice.

THE 2026 WAY

Docs → ChatGPT → code → run

DOCS FILE

the SDK "LLM builder" .md



PROMPT

the strategy in plain words



CODE

one Python file



RUN

orders appear in UAT

What the prompt says, roughly

"Using the attached Nubra Python SDK docs, write one script: log in to UAT, stream 1-minute NIFTY candles over the WebSocket, detect red→green and green→red flips, buy the ATM CE/PE of the nearest expiry, 10 lots, square off the previous leg first, print every event."

The model writes the plumbing. You still have to understand every part, which is what the next five slides are for.



Login once, then listen

```
from nubra_python_sdk.start_sdk import InitNubraSdk, NubraEnv    # the login call + the env switch
from nubra_python_sdk.ticker.websocketdata import NubraDataSocket # the WebSocket client

nubra = InitNubraSdk(NubraEnv.UAT, env_creds=True)              # log in once; phone + MPIN from .env

def on_ohlc_data(msg):                                          # runs on EVERY stream message
    if msg.bucket_timestamp != state["bucket"]:                # a NEW minute started...
        candle_done(state["last"])                             # ...so the previous one is complete
        state["bucket"], state["last"] = msg.bucket_timestamp, msg # remember the candle being built

socket = NubraDataSocket(client=nubra, on_ohlc_data=on_ohlc_data) # hand it the callback
socket.connect()                                                  # open the connection (its own thread)
socket.subscribe(["NIFTY"], data_type="ohlc", interval="1m", exchange="NSE") # the candle feed
```

The stream sends the **current** candle as it builds. When the bucket timestamp changes, the previous candle is finished. No polling, no refresh.

Source: SDK docs, Realtime Data → OHLCV stream (fields: open, high, low, close, bucket_timestamp = candle start; NubraDataSocket lives in nubra_python_sdk.ticker.websocketdata).

The whole strategy is **ten lines**

```
def colour(c):  
    if c.close > c.open: return "GREEN"  
    if c.close < c.open: return "RED"  
    return "FLAT"  
  
def update(prev, last):  
    before, now = colour(prev), colour(last)  
    if before == "RED" and now == "GREEN": return "CE" # flipped up: buy the call  
    if before == "GREEN" and now == "RED": return "PE" # flipped down: buy the put  
    return None
```

one candle in, one word out
closed above where it opened
closed below where it opened
equal: ignored

the last two COMPLETED candles
the earlier word, the newer word
flipped up: buy the call
flipped down: buy the put
no flip, do nothing

Everything else in the file is **plumbing**: getting candles in, getting orders out.

"ATM, nearest expiry" in code

```
md = MarketData(nubra) # market-data helper, reuses the login
chain = md.option_chain("NIFTY", exchange="NSE").chain # the whole chain
expiry = sorted(chain.all_expiries)[0] # nearest 'YYYYMMDD'
chain = md.option_chain("NIFTY", expiry=expiry, exchange="NSE").chain # the chain for THAT expiry

rows = chain.ce if signal == "CE" else chain.pe # calls or puts, per the signal
row = next(o for o in rows if o.strike_price == chain.at_the_money_strike) # the ATM row

ref_id, lot_size = row.ref_id, row.lot_size # what the order needs
qty = 10 * lot_size # 10 lots
```

Underlying in

"NIFTY", never an option symbol

ATM

the SDK tells you:
`at_the_money_strike`

Out

a `ref_id` and a `lot_size`

The order **payload**

```
price = round_to_tick(last_traded_price, tick_size)           # integer PAISE, multiple of tick

result = trader.create_order({                                # one dict = one order (docs: V3)
    "refId": ref_id, "qty": qty, "side": "BUY",                # which, how many, which way
    "deliveryType": "IDAY", "priceType": "LIMIT",             # intraday product, at MY price
    "validityType": "DAY",                                     # good for today only
    "isMultiLeg": False, "executionMode": "ENTRY",            # a plain single-instrument entry
    "entryPrice": price,                                       # the limit price, in paise
    "stratTags": ["bmsit-flip-demo"],                          # one tag, hyphens only
})
order_id = result.orders[0].intentOrderId                     # the id you track from now on
```

Paise

₹125.50 is sent as 12550

Tick size

or the exchange rejects it

Tag

find your orders later by
strategy

Close the old leg, **check the new one**

```
if open_leg:                                # bought by THIS script earlier
    place("SELL", open_leg["leg"], open_leg["qty"])    # square off first

order_id = place("BUY", new_leg, qty)            # then the new side

status = trader.get_order(order_id)[0]           # never assume: read it back
print(status.status, status.filledQty, status.rejectionMsg) # what the exchange decided
```

- ◆ An order is a **request**. The exchange decides. Read the status back.
- ◆ Open legs live in memory: if the script dies, square off in the app.

LIVE

RUN IT.

1 · CANDLE

time, open, close,
colour

2 · FLIP DETECTED

→ buy ATM CE / PE

3 · ORDER SENT

qty, limit price, strike

4 · ORDER STATUS

read back, then the
Nubra app

If the market is quiet: `--replay` feeds the last 60 real candles at one per second.

RECAP

One file, **five parts**



Your project today has the **same five parts**. Different signal.



YOUR PROJECT

A **quant** screener

Watch

14 well-known NSE stocks: daily candles for the history, a WebSocket for right now

Measure

How stretched is each one from its own normal? One number: the **z-score**

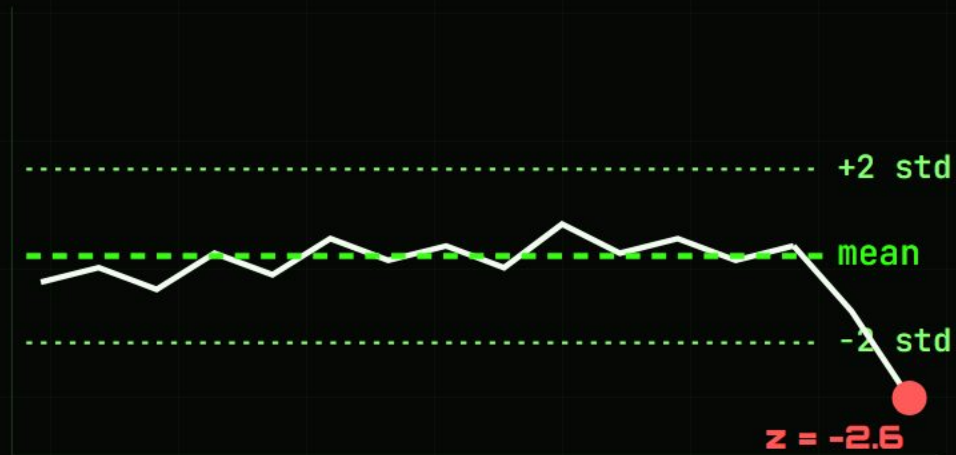
Rank

A table, most stretched first, re-ranking live. Optionally, one sandbox order.

Not "this chart looks oversold". A number, across many stocks, ranked, updating.

MEAN · STANDARD DEVIATION · Z-SCORE

How far from **normal**?



◆ Last 20 closes

One price per trading day, the one at the bell. Twenty of them is about a month. That is the stock's "recent self".

◆ Mean: the centre

Add the 20 closes, divide by 20. The price the stock keeps coming back to. The dashed line.

◆ Std: the usual wobble

How far a typical day sits from the centre. A calm stock has a small std, a jumpy one a big std. About 95% of days land inside the ± 2 band.

◆ $z = (\text{close} - \text{mean}) / \text{std}$

Today's distance from the centre, counted in wobbles. No rupees in it, so a ₹50 stock and a ₹3,000 stock compare. Below -2 is rare: "stretched low", the screener's signal.

THE PROJECT, AS A MACHINE

Seven boxes, in order



History in

`historical_data`, batches of 5 symbols, 1 s apart: 60 calls a minute is the limit. Prices in paise ÷ 100.

Out

symbol · close · mean20 · std20 · z · label, sorted by z. Live, the same table keeps re-ranking.

Live in

The `index` stream, weight 1 per symbol. Every tick replaces today's close, z is recomputed, the table reprints every 2 s.

The order condition

Only when a stock's live z crosses **down** through -2 (was ≥ -2 , now < -2). One order per run, qty 1, LIMIT at LTP rounded to the tick, in UAT. Then read it back.

BOX 6

Risk rules **are** the strategy

- ◆ Max **one** order per run
- ◆ Quantity **1**, always
- ◆ No order unless $z < -2$
- ◆ Never buy a stretched **high**
- ◆ Each refusal prints *why*



HOW YOU BUILD IT

Seven checkpoints. Run after each.

#	CHECKPOINT	YOU SEE	MIN
1	Login to PROD for data (UAT only behind <code>--trade</code>)	"logged in"	3
2	Daily candles for one stock	60 rows of OHLCV in rupees	4
3	The maths for that stock	mean, std, z, label	4
4	Loop the watchlist, rank	the table	5
5	Live: the index WebSocket for the watchlist	the table re-ranking every 2 s	4
6	Risk rules	refusals with reasons	2
7	One UAT order on a -2 cross-down (optional)	order id, status	3

YOUR TURN · ATTACH THE DOCS, TYPE THIS, BUILD FOR 25 MINUTES

Ask for **exactly this**. Then build.

Your prompt

"Using the attached Nubra Python SDK docs, write `screener.py` in seven numbered checkpoints: (1) log in to **PROD** for market data with `env_creds=True`, and to **UAT** only behind a `--trade` flag; (2) fetch 60 daily candles for one NSE stock with `historical_data` and convert paise to rupees; (3) compute the 20-day mean, standard deviation and z-score of the close; (4) loop over this watchlist in batches of 5 with a 1-second pause and print a table sorted by z; (5) subscribe to the realtime `index` stream for the watchlist and, on every update, replace today's close with the live price, recompute z and reprint the table every 2 seconds; (6) risk rules: at most one order per run, quantity 1, only if $z < -2$, never a stretched high, and print why each refusal happens; (7) behind `--trade`, when a stock's live z crosses down through -2 , place one LIMIT BUY in UAT at the last price rounded to the tick size and read the order status back. Comment every step in plain English."

paise ÷ 100

ref_id: resolve, never guess

UTC dates in the request

60 calls / min: batches, pauses

index_value is paise

25:00

START

press again to reset · bell at zero

Pairs

No account? Sit with someone who has one.
Two people, one laptop.

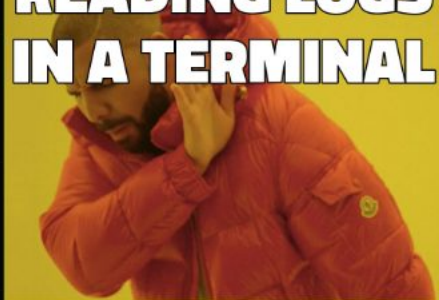
Stuck?

Raise a hand. Volunteers are walking.
Checkpoint 4 is the finish line; 5 to 7 are bonus.

PART TWO

Logs are for machines. **Dashboards are for you.**

READING LOGS
IN A TERMINAL



A LIVE
DASHBOARD



- ◆ What is the strategy doing **right now**?
- ◆ Which stocks are stretched, **at a glance**?
- ◆ One page you can show someone who does not read Python

Remember **Nubra OSS**? The visual dashboards we saw there this morning. Now build one for **your** screener.

THE PROMPT PATTERN

Structure first, then code

1 • Structure

"I want a live dashboard for this screener. What repo structure would you use? List the files and what each one does."

2 • Code

"Now give me the full code for each file, one at a time."

3 • Run

"How do I install and run it?"
Then paste any error back, verbatim.

dashboard/

```
app.py          # the page: table, z-score bars, one price chart with bands
data.py         # reads the cache or the screener's live table
indicators.py   # mean, std, z    (the same maths, in one place)
requirements.txt # whatever the model picked: Streamlit, Dash, Flask, plain HTML...
```

Roughly the shape you will get, whatever the tool. Asking for structure first stops the model from writing one giant file.

LIVE CHATGPT → A DASHBOARD

Ask

structure, then code, then "how do I run it?"

Run

whatever it returns: install it, run it, as it says

Fix

paste every error back, verbatim, until it runs

The tool does not matter. The loop does: ask, run, paste the error, run again.

YOUR TURN • 15 MINUTES • SAME THREE PROMPTS

A page for **your** screener. Live.

1 • Structure

"I want a live dashboard for this screener (attach `screener.py`). It reruns the maths as the live table changes. What repo structure would you use? List the files and what each does."

2 • Code

"Now give me the full code for each file, one at a time."

3 • Run

"How do I install and run it?" Paste every error back, verbatim.

Minimum: the ranked table + a bar chart of z

Bonus: one stock's price with mean and ± 2 bands

Show-off: it re-ranks itself as the stream ticks

15:00

START

press again to reset • bell at zero

Where the live data is

Your screener's live step writes its table every 2 s (ours: `cache/_live.csv`). Read that file; do not open a second stream.

AFTER THE BELL

No login? Here is the finished project.

github.com/aaryan-say/bmsit-quant-101

screener.py

the seven
checkpoints,
commented

dashboard

app, data, indicators:
the page

cache

real daily candles, so
no login is needed

README

how to run it, the five
classic mistakes

```
git clone https://github.com/aaryan-say/bmsit-quant-101.git
cd bmsit-quant-101
cd session2
python screener.py --offline           # checkpoints 2, 3, 4, 6 on cached data
```

SHOWDOWN · ROUND 2

SHOWDOWN ROUND 2

8 questions on what you just built. Fastest correct answer wins.

`backboard-crafter-closure.ngrok-free.dev`

Same link as this morning. Log in with your USN and PIN.

ONE QUESTION LEFT

You built it. **Does it work?**

- ◆ The flip strategy placed orders today. Were they **good** orders?
- ◆ Your screener found stretched stocks. Do they **bounce back**?
- ◆ There is only one honest way to know: **run it on the past.**

Session 3: backtesting. Bring a strategy idea.

